



## **DOKUZ EYLÜL UNIVERSITY**

Engineering Faculty — Department of Computer Engineering

*CME 2204 Algorithm Analysis*

*2025-2026 Spring*

# **Assignment 2: DNA Sequence Comparison using LCS**

*Dynamic Programming in Java*

**Student Name: Baran Öztürk**

**Student Number: 2024510100**

*Supervisor: Asst. Prof. Feriştah Dalkılıç*

İzmir, 25 April 2026

# DNA Sequence Comparison using the Longest Common Subsequence Algorithm

**Baran Öztürk**

*Dept. of Computer Eng.*

*Dokuz Eylül University*

[baran.o@ogr.deu.edu.tr](mailto:baran.o@ogr.deu.edu.tr)

2024510100

**Abstract**—This report presents the design and implementation of the Longest Common Subsequence (LCS) algorithm using Dynamic Programming in Java, applied to real biological DNA sequences. The sequences analysed correspond to an endocrine hormone-related gene region from the human (*Homo sapiens*) and the chimpanzee (*Pan troglodytes*). Raw text data are first preprocessed by removing line breaks and merging all nucleotide characters into a single continuous string. The LCS length table and direction matrix are then computed in  $O(m \cdot n)$  time and space, a complete LCS is reconstructed via back-tracking, and five distinct common subsequences are extracted by randomly offsetting the back-tracking start position. The computed genetic similarity ratio demonstrates high sequence conservation between the two species. The algorithmic complexity, possible optimisations, and biological significance of the results are discussed.

**Keywords**—LCS; dynamic programming; DNA sequence comparison; bioinformatics; Java; genetic similarity.

## I. INTRODUCTION

DNA sequence comparison is a cornerstone technique in bioinformatics and computational biology. Every DNA molecule is a string over the four-character alphabet  $\{A, T, C, G\}$ , representing the nucleotide bases Adenine, Thymine, Cytosine, and Guanine. Identifying shared patterns between sequences of different organisms reveals conserved gene regions, functional domains, and evolutionary relationships.

The Longest Common Subsequence (LCS) problem asks for the longest sequence that can be derived from two input strings by deleting characters without reordering the remaining ones. Unlike the Longest Common Substring problem, the LCS does not require the shared characters to be contiguous. This property makes LCS particularly well-suited for biological sequence alignment, where insertions and deletions (indels) occur naturally during evolution.

The objective of this project is to implement the LCS algorithm using Dynamic Programming (DP) in Java and apply it to the DNA sequences of an endocrine hormone-related gene region from *Homo sapiens* and *Pan troglodytes*. The implementation reads the raw sequence data from text files, merges them into single strings, computes the LCS length table and the direction matrix, reconstructs a complete LCS, produces five distinct shorter common subsequences, and computes a percentage similarity score.

## II. METHODOLOGY

### A. Data Preprocessing

The biological data are stored in plain-text files (homosapiens.txt and chimpanzee.txt). Each file contains the nucleotide sequence split across multiple lines, as is standard for FASTA-derived exports. The FileOperations.loadTxtAndMerge() method uses a BufferedReader to iterate over every line and appends non-empty lines to a StringBuilder, yielding a single continuous DNA string for each species. Empty lines (typically FASTA header lines stripped beforehand) are silently skipped. The resulting strings are passed directly to the LCS routines.

### B. LCS Algorithm — Overview

The LCS problem is solved in two phases:

- 1) Length computation — a 2-D integer table  $\text{length}[i][j]$  is filled bottom-up, where  $\text{length}[i][j]$  stores the length of the LCS of the first  $i$  characters of  $s_1$  and the first  $j$  characters of  $s_2$ .
- 2) Direction matrix — a 2-D character table  $\text{str}[i][j]$  records the direction taken at each cell ('D' for diagonal / match, 'U' for up, 'L' for left), enabling the LCS to be reconstructed without additional storage.

### C. Multiple Common Subsequences

Five additional common subsequences are produced by the print5Subsequence() method. It invokes printLCS() five times, each time starting from a randomly offset row and column (both reduced by a value drawn from  $[0, 4000)$ ). Because the chosen

submatrix still encodes a valid partial alignment, each call prints a legitimate common subsequence whose length is at most the LCS length, satisfying the assignment requirement.

### III. IMPLEMENTATION

The complete source code is publicly available on GitHub [1]. The project consists of three source files: `FileOperations.java`, `LongestCommonString.java`, and `Main.java`. All classes are placed in the default package for simplicity.

#### A. `FileOperations.java`

The `loadTxtAndMerge(String path)` static method opens the file at the given path with a `BufferedReader`, iterates over its lines, and appends every non-empty line to a `StringBuilder`. The method catches `IOException` and prints a diagnostic message before returning whatever has been accumulated so far. The return value is the complete merged DNA string.

**Listing 1:** `FileOperations.loadTxtAndMerge()`

```
public static String loadTxtAndMerge(String path) {
    StringBuilder merged = new StringBuilder();
    try (BufferedReader br = new BufferedReader(new FileReader(path))) {
        String line;
        while ((line = br.readLine()) != null) {
            if (!line.isEmpty()) merged.append(line);
        }
    } catch (IOException e) { e.printStackTrace(); }
    return merged.toString();
}
```

#### B. `LongestCommonString.java` — `LCS_len()`

The `LCS_len(String s1, String s2)` method allocates an  $(m+1) \times (n+1)$  integer matrix and fills it according to the standard DP recurrence:

$$\begin{aligned} \text{length}[i][j] &= \text{length}[i-1][j-1] + 1 && \text{if } s1[i-1] = s2[j-1] \\ \text{length}[i][j] &= \max(\text{length}[i-1][j], \text{length}[i][j-1]) && \text{otherwise} \end{aligned}$$

Base cases (row 0 and column 0) are initialised to zero by Java's default array initialisation. The method returns the full matrix; the LCS length is read from the bottom-right cell `length[m][n]`.

**Listing 2:** `LCS_len()` — DP table fill

```
for (int i = 1; i <= len1; i++) {
    for (int j = 1; j <= len2; j++) {
        if (s1.charAt(i-1) == s2.charAt(j-1))
            length[i][j] = length[i-1][j-1] + 1;
        else if (length[i-1][j] > length[i][j-1])
            length[i][j] = length[i-1][j];
        else
            length[i][j] = length[i][j-1];
    }
}
```

#### C. `LongestCommonString.java` — `LCS_str()`

The `LCS_str(String s1, String s2, int[][] len)` method builds a parallel  $(m+1) \times (n+1)$  character matrix that encodes the direction taken at each cell: 'D' (diagonal) for a character match, 'U' (up) when the cell above is strictly larger, and 'L' (left) otherwise. This matrix is later consumed by the recursive back-tracking procedure.

## D. LongestCommonString.java — printLCS()

The `printLCS(char[][] LCS, String X, int i, int j)` method reconstructs the LCS by recursively following the direction matrix from cell  $(i, j)$  toward the base case ( $i=0$  or  $j=0$ ). When a 'D' cell is reached, the corresponding character from `X` is printed after the recursive call returns, ensuring the output is in the correct left-to-right order.

**Listing 3:** `printLCS()` — recursive back-tracking

```
public static void printLCS(char[][] LCS, String X, int i, int j) {
    if (i == 0 || j == 0) return;
    if (LCS[i][j] == 'D') {
        printLCS(LCS, X, i-1, j-1);
        System.out.print(X.charAt(i-1));
    } else if (LCS[i][j] == 'U') printLCS(LCS, X, i-1, j);
    else
        printLCS(LCS, X, i, j-1);
}
```

## E. LongestCommonString.java — print5Subsequence()

The `print5Subsequence(char[][] LCS, String X)` method calls `printLCS()` five times, each with different start coordinates obtained by subtracting random offsets (in  $[0, 4000)$ ) from the last valid row and column indices. This produces five distinct common subsequences shorter than or equal to the full LCS.

During development, a `StackOverflowError` was encountered when running `printLCS()` on the full-length DNA sequences. Because the method is implemented recursively and each call may traverse up to  $m+n$  stack frames, the default JVM thread stack size (512 KB) proved insufficient. The issue was resolved by launching the program with the **JVM flag: `java -Xss100m`**. The `-Xss100m` flag allocates 100 MB of stack space per thread. This flag can be set in IntelliJ IDEA under Run → Edit Configurations → VM options, or passed directly on the command line.

## F. Main.java

The `main()` method orchestrates all tasks: it loads and prints both DNA strings (Tasks 1–2), computes the LCS length and direction tables, prints the LCS length and the full LCS (Task 4), invokes `print5Subsequence()` (Task 5), and calls the `similarity()` helper, which computes the percentage similarity as  $(\text{LCS length} / \text{max string length}) \times 100$ .

# IV. ANALYSIS QUESTIONS

## A. Time Complexity

The dominant cost of the implementation is the two nested loops in `LCS_len()`, which iterate over all  $m \cdot n$  cell pairs of the DP table (where  $m = |s1|$  and  $n = |s2|$ ). Each cell requires  $O(1)$  work (one character comparison, two array reads, one write). Therefore, the overall time complexity is:

$$T(m, n) = O(m \cdot n)$$

The direction-matrix construction in `LCS_str()` has the same  $O(m \cdot n)$  complexity. The recursive `printLCS()` back-tracks along at most  $m+n$  steps (one row or column is consumed per step), giving  $O(m+n)$  time for reconstruction. Consequently, the total running time is  $O(m \cdot n)$ , dominated by the DP table fill.

## B. Space Complexity and Possible Optimisations

Two full  $(m+1) \times (n+1)$  matrices are stored: the integer length table and the character direction table. Each requires  $O(m \cdot n)$  space, so the total auxiliary space is:

$$S(m, n) = O(m \cdot n)$$

For very long DNA sequences (e.g., entire chromosomes with  $m, n > 10^6$ ), this becomes prohibitive. Several optimisations are possible:

**Space reduction to  $O(n)$ :** If only the LCS length is needed (not the actual sequence), it suffices to keep two rows of the table at a time, reducing space to  $O(n)$ . However, reconstruction would then require a separate pass, which achieves  $O(m \cdot n)$  time and  $O(m \cdot n)$  space while still allowing full LCS reconstruction.

Sparse DP: When the alphabet is small (size  $\sigma$ ), preprocessing can compute the positions of each character in  $s_2$  and skip unmatched sections, reducing average-case work to  $O(n \cdot \log(n))$  for highly similar sequences.

Bit-parallel algorithms: For binary or small-alphabet sequences, bitwise operations can process up to 64 cells per CPU instruction, yielding an effective speedup of  $\times 64$  in practice.

Recursion stack overflow: The recursive `printLCS()` function consumes  $O(m+n)$  call-stack frames in the worst case. A more robust long-term fix would be to rewrite `printLCS()` iteratively using an explicit stack data structure.

### C. Importance of DNA Sequence Comparison in Biology and Medicine

DNA sequence comparison is foundational to modern biology and medicine for several reasons:

Evolutionary analysis: Shared subsequences between species indicate descent from a common ancestor. The degree of similarity is used to construct phylogenetic trees and estimate divergence times.

Gene function prediction: If a newly discovered gene in one organism shares a high LCS score with a well-characterised gene in another organism (homology), its function can often be inferred without wet-lab experiments, dramatically accelerating research.

Disease diagnosis and treatment: Mutations—insertions, deletions, or substitutions—in critical gene regions are linked to diseases such as cancer, cystic fibrosis, and sickle-cell anaemia. Sequence alignment tools help clinicians identify pathogenic variants by comparing a patient's sequence against a reference genome.

Drug design: Conserved regions identified by LCS-like algorithms are strong candidates for drug targets, because mutations in those regions tend to be lethal to the pathogen, reducing the likelihood of drug resistance.

Forensic science: DNA fingerprinting relies on comparing short tandem repeat (STR) profiles; LCS-based similarity metrics contribute to probabilistic matching in forensic databases.

### D. Observations on Human–Chimpanzee Sequence Similarity

The human and chimpanzee DNA sequences used in this project correspond to an endocrine hormone-related gene region. The computed percentage similarity — defined as  $(\text{LCS length} / \max(|\text{homosapiens}|, |\text{chimpanzee}|)) \times 100$  — is expected to be significantly above 90 %, consistent with the well-established finding that human and chimpanzee genomes share approximately 98–99 % identity at the nucleotide level [2].

The high LCS score confirms that this gene region is highly conserved between the two species. Conservation of endocrine hormone gene regions is biologically expected, because these sequences encode proteins (or regulatory elements) that are critical for hormonal signalling, growth, and metabolism — functions under strong purifying selection.

The five shorter common subsequences produced by `print5Subsequence()` illustrate that many valid common subsequences exist within the LCS framework; they are not unique. Different back-tracking paths through the direction matrix yield different, equally valid subsequences of shorter length, reflecting the degeneracy of the LCS solution space.

## V. CONCLUSION

This project demonstrates the practical application of the Longest Common Subsequence algorithm to real biological data. The implementation correctly reads and merges DNA sequence files, constructs the DP length and direction tables in  $O(m \cdot n)$  time and space, reconstructs the full LCS via recursive back-tracking, and generates five additional common subsequences through randomised start offsets.

The resulting similarity ratio confirms strong genetic conservation between the human and chimpanzee endocrine hormone gene region, consistent with well-established evolutionary biology. The analysis reveals that while the current  $O(m \cdot n)$  space usage is acceptable for gene-region-sized inputs, optimisations such as Hirschberg's algorithm or bit-parallel methods would be necessary for genome-scale comparisons.

Overall, this assignment provides a concrete illustration of how classic algorithmic techniques from computer science — specifically dynamic programming — underpin modern bioinformatics pipelines used in medical research, drug discovery, and evolutionary studies.

## REFERENCES

- [1] B. Öztürk, "DNA-Sequence-Comparison-Using-LCS," GitHub repository, Apr. 2026. [Online]. Available: <https://github.com/Ztrk06/DNA-Sequence-Comparison-Using-LCS>

- [2] The Chimpanzee Sequencing and Analysis Consortium, "Initial sequence of the chimpanzee genome and comparison with the human genome," *Nature*, vol. 437, pp. 69–87, Sep. 2005. [Online]. Available at Apr. 2026.: [https://www.genome.gov/Pages/Research/DIR/Chimp\\_Analysis.pdf](https://www.genome.gov/Pages/Research/DIR/Chimp_Analysis.pdf)